

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

E.P. de Ingeniería de Telecomunicaciones



Entregable 2:

Analizador inteligente de ocupación en bandas ISM mediante sistema digital con memoria direccionada y modelo TinyML para selección adaptativa de canal

DOCENTE:

Villafuerte Barreto, Hernán Oswaldo

CURSO:

Circuitos Digitales

INTEGRANTES:

Agelvis Vargas, Gustavo Enrique

Espinoza Pagola, Frank Abner

Izarra Gamarra, Annabeth Valeria

Vallejos Terrones, Brenda Lucero

Lima, 2026

1. Propuesta de Proyecto

1.1. Descripción

El proyecto implementa un sistema digital capaz de analizar métricas de ocupación de canales WiFi en la banda ISM, almacenar información en memoria direccionada y seleccionar automáticamente el canal menos congestionado mediante una FSM y lógica de decisión LUT implementadas en FPGA.

1.2. Materiales

- FPGA DE2-115 (Cyclone IV E)
- ESP32-S3
- Laptop con Quartus Prime y ModelSim
- Cable USB-Blaster
- Protoboard y cables Dupont
- Fuente de alimentación
- Router WiFi para pruebas

1.3. Objetivos

1.3.1. General

Desarrollar un sistema digital inteligente para análisis y selección adaptativa de canales en banda ISM utilizando FPGA.

1.3.2. Específicos

- Diseñar una FSM para controlar el flujo del sistema.
- Implementar memoria direccionada y decodificador de direcciones.
- Validar mediante simulación funcional la selección automática del canal menos congestionado.

1.4. Caso de uso

En entornos con múltiples redes WiFi, el sistema analiza el nivel de ocupación de los canales disponibles y recomienda automáticamente el canal con menor congestión para mejorar la calidad de comunicación.

1.5. Diagrama de Bloques

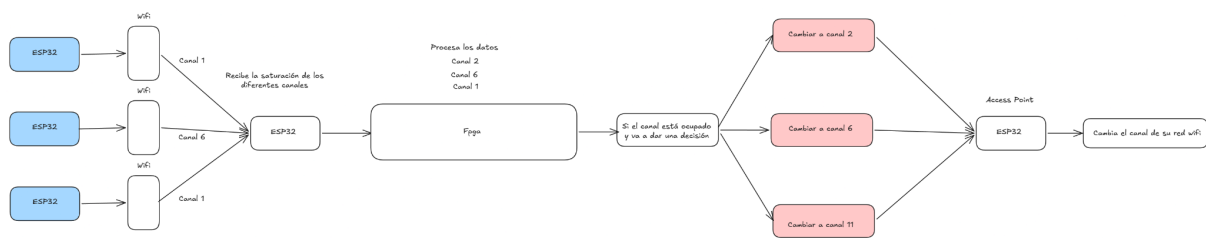


Figura 1.- Diagrama general de bloques del sistema

2. Diseño lógico y FSM del sistema

2.1. Descripción funcional del sistema

El sistema implementa una arquitectura digital basada en FPGA para el análisis y selección adaptativa de canales en bandas ISM. La solución utiliza tres módulos ESP32 configurados para monitorear distintos canales WiFi, en este caso los 1, 6 y 11. Cada módulo realiza mediciones de ocupación del canal asignado y transmite dicha información hacia un ESP32 concentrador.

El ESP32 concentrador recopila las métricas de los tres canales y genera una trama de entrada dirigida al FPGA. Esta trama contiene principalmente el identificador del canal y su nivel de ocupación normalizado en un rango decimal entre 0 y 1, representando el porcentaje de saturación del medio inalámbrico.

Dentro del FPGA, la información recibida es procesada mediante una lógica secuencial controlada por una máquina de estados finitos (FSM). Inicialmente, los datos de entrada son separados y organizados en parámetros individuales: canal, RSSI y ocupación. Posteriormente, el valor de ocupación es evaluado mediante una LUT de decisión que clasifica el estado del canal, según rangos predefinidos, en tres categorías: libre (0-30%), moderado (31-70%) o congestionado (71-100%).

Las métricas procesadas son almacenadas en una memoria direccionada interna, permitiendo conservar el estado de cada canal para posteriores operaciones de análisis. Una vez completada la escritura de datos, el sistema avanza hacia un estado de comparación, donde se evalúan los niveles de ocupación almacenados para determinar cuál de los canales monitoreados presenta menor congestión.

Finalmente, el resultado de la comparación es escrito en un registro de salida y enviado hacia un ESP32 encargado de operar como punto de acceso WiFi. La decisión del canal menos

ocupado es realizada completamente por el FPGA mediante el procesamiento de las métricas almacenadas y evaluadas internamente por la lógica digital del sistema. El ESP32 de salida utiliza la información proporcionada por el FPGA únicamente para ejecutar el cambio dinámico hacia el canal seleccionado. De esta manera, el sistema integra captura distribuida, procesamiento digital secuencial, memoria direccionada y lógica de decisión adaptativa para optimizar la selección de canal en redes inalámbricas.

2.2. Arquitectura del sistema

La arquitectura del sistema se encuentra dividida en tres etapas principales: adquisición de métricas de ocupación, procesamiento digital en FPGA y reconfiguración adaptativa del canal WiFi. El sistema emplea módulos distribuidos ESP32 para la captura de información de ocupación, mientras que la FPGA centraliza el procesamiento secuencial, almacenamiento de métricas y selección del canal óptimo.

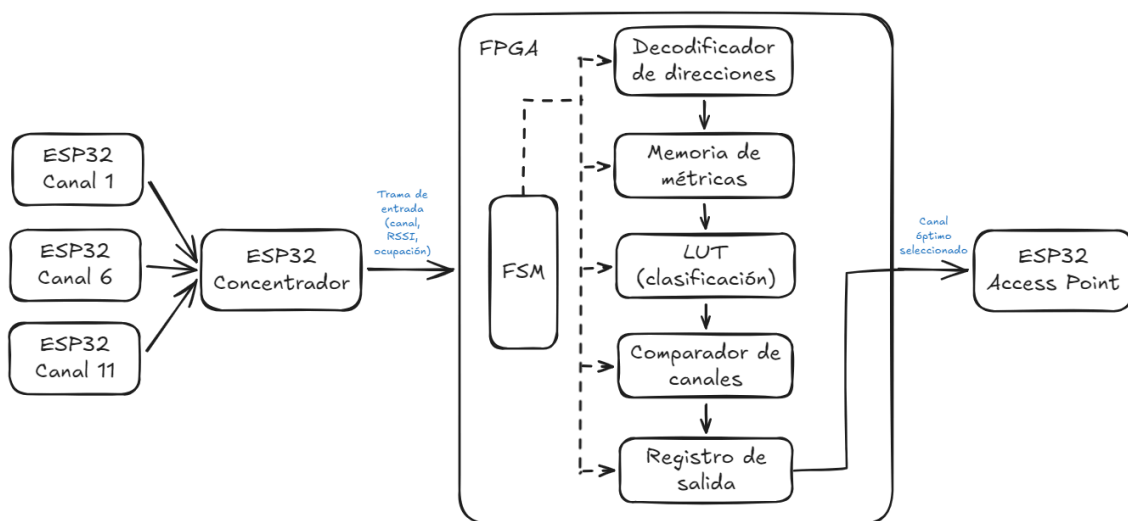
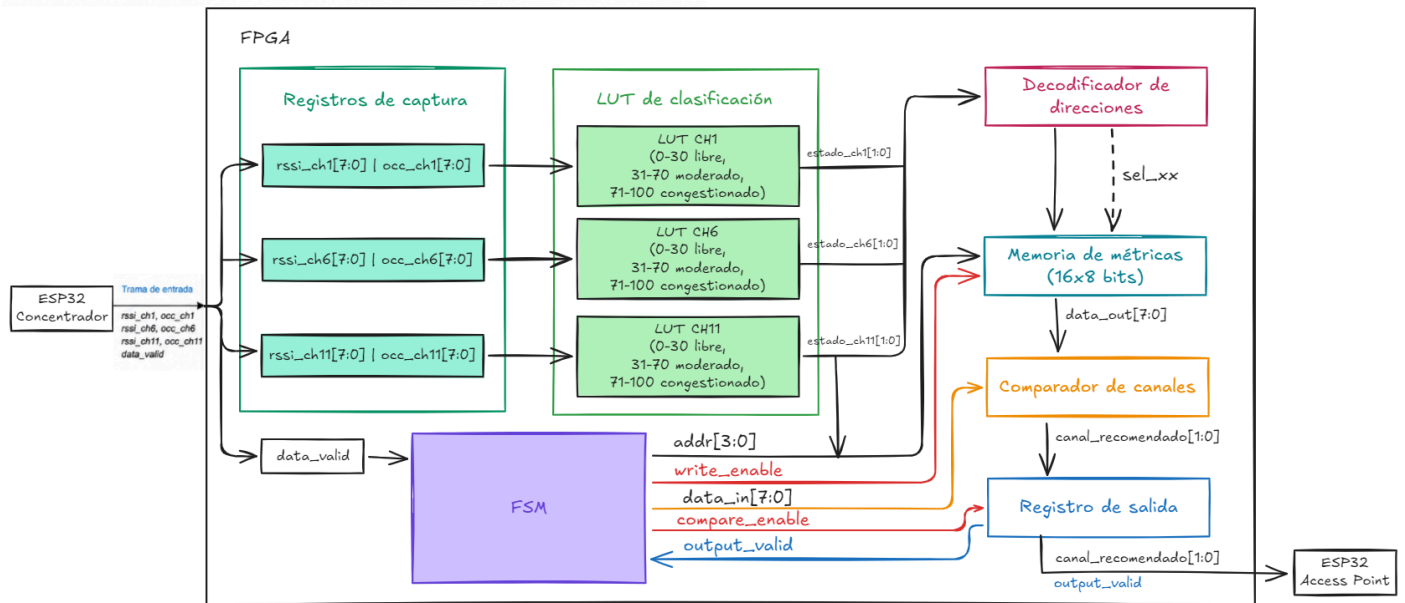


Figura 2.-Diagrama de arquitectura del sistema

2.3. Diagrama lógico detallado

El procesamiento del sistema se realiza de manera secuencial dentro de la FPGA mediante una FSM encargada de coordinar las etapas de captura, clasificación, almacenamiento y comparación de métricas de ocupación inalámbrica. Inicialmente, las métricas provenientes del ESP32 concentrador son registradas internamente para estabilizar los datos de entrada. Posteriormente, las ocupaciones de los canales son evaluadas mediante el LUT de clasificación que determina el estado de cada canal según su nivel de congestión.

Una vez obtenidos los estados correspondientes, la FSM genera las señales de control necesarias para direccionar la memoria y almacenar las métricas asociadas a cada canal.



Finalmente, el sistema ejecuta la comparación de ocupaciones almacenadas para seleccionar el canal menos saturado y enviar el resultado hacia el ESP32 encargado de reconfigurar dinámicamente el punto de acceso WiFi.

Figura 3: Diagrama lógico detallado de la solución: procesamiento en el FPGA

2.4. Lógica del mapa de memoria

La memoria interna del sistema se organiza mediante posiciones direccionadas dedicadas al almacenamiento de las métricas asociadas a cada canal analizado. Cada canal dispone de tres posiciones reservadas para RSSI, ocupación y estado clasificado. Adicionalmente, se incluye una dirección destinada al almacenamiento del canal recomendado generado por el comparador.

Esta organización permite que la FSM acceda secuencialmente a las métricas utilizando direcciones específicas durante las etapas de escritura y lectura del sistema.

Dirección	Contenido
0x00	RSSI canal 1
0x01	Ocupación canal 1
0x02	Estado clasificado canal 1
0x04	RSSI canal 6
0x05	Ocupación canal 6
0x06	Estado clasificado canal 6
0x08	RSSI canal 11

0x09	Ocupación canal 11
0x0A	Estado clasificado canal 11
0x0F	Canal recomendado (resultado final)

Tabla 1.-Mapa de memoria del sistema

2.5. Lógica de direccionamiento

El direccionamiento de memoria del sistema es controlado secuencialmente por la FSM principal mediante la señal $\text{addr}[3:0]$. Durante cada estado de escritura, la máquina de estados asigna una dirección específica asociada a la métrica correspondiente del canal analizado, permitiendo almacenar valores de RSSI, ocupación y estado clasificado dentro de la memoria interna.

Este mecanismo organiza lógicamente la información de cada canal y facilita el acceso estructurado a las métricas durante la etapa de comparación.

Estado FSM	Dirección generada	Operación realizada
WRITE CH1 RSSI	0x00	Escritura de RSSI del canal 1
WRITE CH1 OCC	0x01	Escritura de ocupación del canal 1
WRITE CH1 STATE	0x02	Escritura del estado del canal 1
WRITE CH6 RSSI	0x04	Escritura de RSSI del canal 6
WRITE CH6 OCC	0x05	Escritura de ocupación del canal 6
WRITE CH6 STATE	0x06	Escritura del estado del canal 6
WRITE CH11 RSSI	0x08	Escritura de RSSI del canal 11
WRITE CH11 OCC	0x09	Escritura de ocupación del canal 11
WRITE CH11 STATE	0x0A	Escritura del estado del canal 11
WRITE RESULT	0x0F	Escritura del canal recomendado

Tabla 2.- Lógica de direccionamiento

2.6. Lógica del decodificador de direcciones

El decodificador de direcciones es el bloque encargado de interpretar la dirección generada por la FSM principal y habilitar la posición lógica correspondiente dentro de la memoria. A partir de la señal $\text{addr}[3:0]$, el sistema activa únicamente la línea asociada a la métrica seleccionada, permitiendo organizar y acceder de forma estructurada a la información almacenada de cada canal.

Durante los estados de escritura, la FSM asigna direcciones específicas para RSSI, ocupación y estado clasificado de los canales analizados. Posteriormente, el decodificador habilita internamente la posición correspondiente para realizar la operación de almacenamiento.

DIRECCIÓN (hex)	SEÑAL ACTIVADA	MÉTRICA ASOCIADA
0x00	sel rssi ch1	RSSI del canal 1
0x01	sel occ ch1	Ocupación del canal 1
0x02	sel state ch1	Estado del canal 1
0x04	sel rssi ch6	RSSI del canal 6
0x05	sel occ ch6	Ocupación del canal 6
0x06	sel state ch6	Estado del canal 6
0x08	sel rssi ch11	RSSI del canal 11
0x09	sel occ ch11	Ocupación del canal 11
0x0A	sel state ch11	Estado del canal 11
0x0F	sel recommended	Canal recomendado

Tabla 3.-Mapa de decodificación

2.7. Diseño de la FSM

La FSM propuesta sigue una secuencia lineal de catorce estados, desde la espera inicial hasta la entrega del canal recomendado. Esta estructura permite que cada operación del sistema ocurra en un ciclo controlado: captura de datos, escritura de métricas, clasificación del estado de cada canal, comparación de ocupación y generación de salida. De esta forma, la FSM actúa como unidad de control principal del sistema digital implementado en FPGA.

La FSM implementada corresponde a una máquina de Moore, debido a que las señales de control y salidas del sistema dependen únicamente del estado actual de la máquina y no directamente de las entradas externas. Cada estado activa acciones específicas, como escritura en memoria, comparación o generación de salida, garantizando un comportamiento secuencial estable y sincronizado con el reloj del sistema.

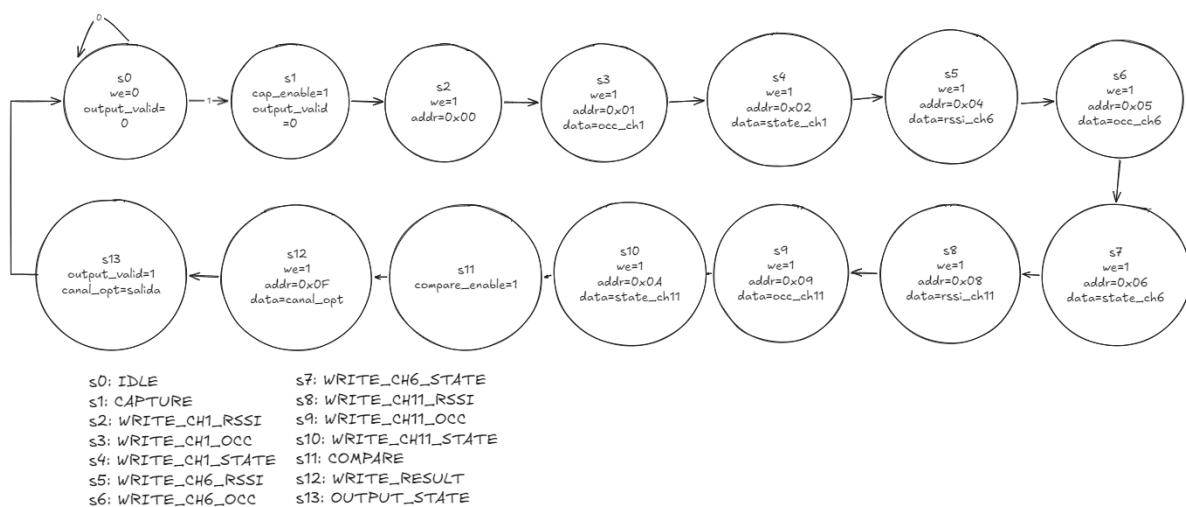


Figura 4: Diagrama de transiciones de la Máquina de Estados Finitos (FSM)

Código	Estado	Función	Siguiente estado
0	IDLE	Espera una trama válida	CAPTURE si data_valid = 1
1	CAPTURE	Captura RSSI y ocupación de los canales 1, 6 y 11	WRITE_CH1_RSSI
2	WRITE_CH1_RSSI	Escribe RSSI del canal 1 en memoria	WRITE_CH1_OCC
3	WRITE_CH1_OCC	Escribe ocupación del canal 1	WRITE_CH1_STATE
4	WRITE_CH1_STATE	Escribe estado clasificado del canal 1	WRITE_CH6_RSSI
5	WRITE_CH6_RSSI	Escribe RSSI del canal 6 en memoria	WRITE_CH6_OCC
6	WRITE_CH6_OCC	Escribe ocupación del canal 6	WRITE_CH6_STATE
7	WRITE_CH6_STATE	Escribe estado clasificado del canal 6	WRITE_CH11_RSSI
8	WRITE_CH11_RSSI	Escribe RSSI del canal 11 en memoria	WRITE_CH11_OCC
9	WRITE_CH11_OCC	Escribe ocupación del canal 11	WRITE_CH11_STATE
10	WRITE_CH11_STATE	Escribe estado clasificado del canal 11	COMPARE
11	COMPARE	Compara ocupaciones y obtiene el canal menos ocupado	WRITE_RESULT
12	WRITE_RESULT	Escribe el canal recomendado en memoria	OUTPUT STATE
13	OUTPUT STATE	Activa output_valid y entrega el resultado	IDLE

Tabla 4.- Tabla de estados de la FSM

2.8. Diagrama ASM

El diagrama ASM (Algorithmic State Machine) muestra la secuencia operacional de la FSM, indicando las acciones realizadas en cada estado y la única decisión inicial asociada a la validez de la trama de entrada (data_valid). Una vez validada la entrada, el sistema ejecuta una secuencia determinística de captura, escritura de métricas, comparación y generación de salida.

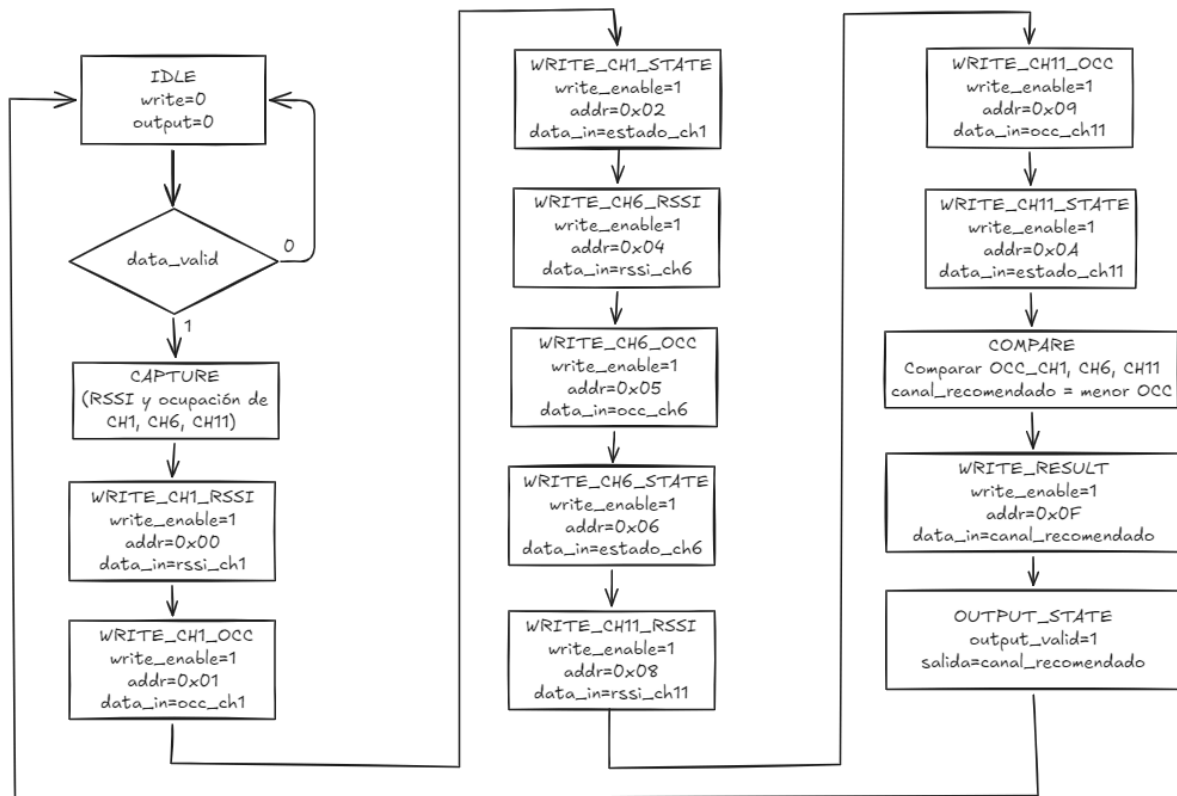


Figura 5: Carta ASM para el flujo de control secuencial.

2.9. Tabla de señales de control por estado

La tabla a continuación muestra las señales principales generadas por la FSM en cada estado. Al tratarse de una máquina de Moore, las señales de control dependen únicamente del estado actual, permitiendo coordinar de forma secuencial la captura, escritura en memoria, comparación y validación de salida.

Estado	write_en able	addr[3:0]	data_in	output_val id	Acción principal
IDLE	0	-	-	0	Espera data valid = 1
CAPTURE	0	-	-	0	Captura simultánea de métricas en registros internos
WRITE_CH1_RSSI	1	0x00	rssi_ch1	0	Escribe RSSI del canal 1
WRITE_CH1_OCC	1	0x01	occ_ch1	0	Escribe ocupación del canal 1
WRITE_CH1_STATE	1	0x02	state_ch1	0	Escribe estado clasificado del canal 1
WRITE_CH6_RSSI	1	0x04	rssi_ch6	0	Escribe RSSI del canal 6
WRITE_CH6_OCC	1	0x05	occ_ch6	0	Escribe ocupación del canal 6
WRITE_CH6_STATE	1	0x06	state_ch6	0	Escribe estado clasificado del canal 6
WRITE_CH11_RSSI	1	0x08	rssi_ch11	0	Escribe RSSI del canal 11

WRITE_CH11_OCC	1	0x09	occ_ch11	0	Escribe ocupación del canal 11
WRITE_CH11_STATE	1	0x0A	state_ch11	0	Escribe estado clasificado del canal 11
COMPARE	0	-	-	0	Compara ocupaciones y obtiene canal óptimo
WRITE_RESULT	1	0x0F	canal_recomendado	0	Escribe canal recomendado en memoria
OUTPUT STATE	0	-	-	1	Indica que la salida está lista

Tabla 5.-Señales de control por estado

3. Código HDL y pseudocódigo

3.1. Estructura modular

El diseño del analizador multicanal de ocupación en banda ISM se organiza de forma modular, de modo que cada componente cumple una función específica e independiente dentro del sistema. Esta sección presenta los cinco módulos que conforman la arquitectura digital: el módulo LUT para la decisión, el módulo decodificador de direcciones, el módulo de memoria de métricas, el módulo comparador de canales y el módulo principal integrado. Para cada uno de ellos se describe su función, se presentan sus entradas y salidas en forma tabular, se incluye su pseudocódigo, su diagrama de bloque y el código HDL correspondiente en Verilog.

La integración de estos módulos permite que la FPGA reciba métricas provenientes de los canales Wi-Fi 1, 6 y 11 —representados por valores de RSSI y porcentaje de ocupación—, las clasifique automáticamente, las almacene en una memoria direccionada y emita una recomendación determinística del canal con menor congestión. El diseño fue desarrollado sobre la plataforma FPGA DE2-115 con dispositivo Intel-Altera Cyclone IV E, utilizando Quartus Prime Lite y ModelSim para simulación.

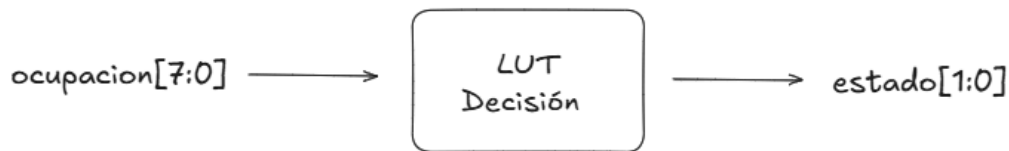
3.2. Módulo LUT para la decisión

El módulo LUT para la decisión tiene como función clasificar el nivel de ocupación de un canal inalámbrico en una de tres categorías: libre, moderado o congestionado. Su operación es completamente combinacional, lo que significa que la salida se actualiza de forma inmediata ante cualquier cambio en la entrada, sin depender de un reloj. El módulo recibe como entrada un valor de 8 bits que representa el porcentaje de ocupación del canal, y entrega como salida un código de 2 bits que indica el estado clasificado.

La clasificación se realiza mediante umbrales fijos: si la ocupación es igual o inferior al 30 %, el canal se considera libre; si se encuentra entre el 31 % y el 70 %, se clasifica como moderado; y si supera el 70 %, se considera congestionado. El código de 2 bits reservado (11)

no se utiliza en esta versión del sistema. Este módulo es instanciado tres veces dentro del módulo principal, una por cada canal analizado.

➤ **Diagrama de bloques**



Codificación de salida:

00 = Libre	(0 % – 30 %)
01 = Moderado	(31 % – 70 %)
10 = Congestionado	(71 % – 100 %)
11 = Reservado	(no utilizado)

Figura 6: Diagrama de bloque de Módulo LUT para la decisión

El diagrama de bloques del módulo LUT para la decisión representa un sistema combinacional encargado de clasificar el nivel de ocupación de un canal inalámbrico. La entrada principal corresponde al valor *ocupacion*, representado mediante 8 bits, el cual ingresa al bloque de decisión para ser evaluado internamente según los rangos establecidos en el diseño. A partir de esta evaluación, el módulo genera una salida denominada *estado*, codificada en 2 bits, que indica la condición del canal analizado.

El flujo mostrado en el diagrama evidencia que el módulo no requiere señales de reloj ni almacenamiento temporal, debido a que su funcionamiento es completamente combinacional. Esto permite que cualquier modificación en el valor de ocupación produzca inmediatamente una actualización en la salida correspondiente. La estructura implementada permite clasificar automáticamente el canal como libre, moderado o congestionado según el porcentaje de ocupación recibido.

Asimismo, el diagrama muestra que este módulo actúa como una unidad de decisión independiente dentro del sistema principal, permitiendo que cada canal WiFi analizado sea procesado de forma individual antes de almacenar el resultado en memoria. Esta organización modular facilita la integración del bloque dentro de la arquitectura general implementada en FPGA.

➤ **Entrada y Salidas**

SEÑAL	ANCHO	DIRECCIÓN	DESCRIPCIÓN
ocupacion	8 bits	Entrada	Porcentaje de ocupación del canal (0 a 100)
estado	2 bits	Salida	00 = Libre 01 = Moderado 10 = Congestionado

Tabla 6.- Entrada y Salidas Módulo LUT para la decisión

➤ **Pseudocódigo****MÓDULO:** decision_lut**ENTRADA:** ocupacion [7:0]**SALIDA:** estado [1:0]

PROCESO COMBINACIONAL:

Si ocupacion $\leq 30 \rightarrow$ estado = 00 (Libre)Si ocupacion $\leq 70 \rightarrow$ estado = 01 (Moderado)Si ocupacion $> 70 \rightarrow$ estado = 10 (Congestionado)*Interpretación:*

El pseudocódigo evidencia una clasificación jerárquica basada en rangos de ocupación. El sistema evalúa primero si el porcentaje es menor o igual al 30 %, asignando el estado de canal libre. Si esta condición no se cumple, se realiza una segunda evaluación hasta el 70 %, clasificando el canal como moderado. Finalmente, cualquier valor superior es considerado congestionado. La estructura garantiza que únicamente una categoría pueda activarse para cada dato de entrada.

➤ **Código HDL (Verilog)**

```

module decision_lut(
    input [7:0] ocupacion,
    output reg [1:0] estado
);
    // 00 = Libre
    // 01 = moderado
    // 10 = congestionado
    // 11 = reservado

    always @(*) begin
        if (ocupacion <= 8'd30)
            estado = 2'b00;
        else if (ocupacion <= 8'd70)
            estado = 2'b01;
        else
            estado = 2'b10;
        end
    endmodule

```

Interpretación:

El código HDL implementa una lógica combinacional mediante el bloque always @(*), lo que permite que cualquier cambio en la entrada ocupacion actualice inmediatamente la salida estado. La estructura condicional if-else establece prioridades de evaluación secuencial, evitando ambigüedades en la clasificación del canal. Además, el uso de constantes binarias facilita la representación digital de cada estado del sistema.

3.3. Módulo decodificador de direcciones

El módulo decodificador de direcciones es un bloque combinacional cuya función es interpretar la dirección de 4 bits generada por la FSM principal y activar exactamente una señal de selección que corresponde a la posición lógica de memoria asociada. De esta forma, el sistema evidencia de manera explícita el circuito combinacional que actúa como interfaz entre el controlador y la memoria, separando la lógica de decodificación de la lógica de almacenamiento.

El módulo acepta diez direcciones válidas, cada una vinculada a una métrica específica del sistema: RSSI, ocupación y estado para cada uno de los tres canales (1, 6 y 11), más una posición reservada para el canal recomendado. Ante una dirección válida, solo la señal de selección correspondiente se pone a nivel alto; el resto permanece en cero. Cualquier dirección no contemplada activa el caso por defecto y mantiene todas las salidas desactivadas.

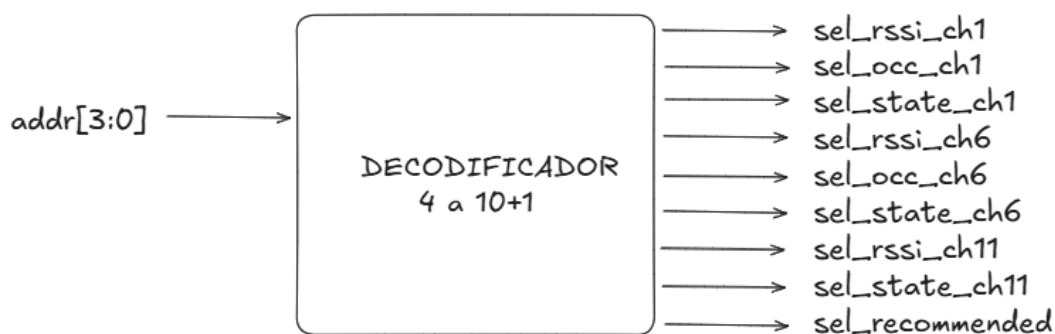
➤ Diagrama de bloques

Figura 7: Diagrama de bloque de Módulo decodificador de direcciones

El diagrama de bloques del decodificador de direcciones representa el mecanismo encargado de traducir la dirección generada por la FSM principal en señales de selección específicas para la memoria del sistema. La entrada *addr*, compuesta por 4 bits, ingresa al bloque decodificador y es evaluada internamente para determinar cuál de las salidas debe activarse.

El funcionamiento mostrado en el diagrama evidencia que únicamente una señal de selección puede permanecer activa a la vez. Cada salida corresponde a una posición lógica asociada al almacenamiento de métricas específicas, como RSSI, ocupación y estado clasificado de los canales 1, 6 y 11, además de la posición reservada para el canal recomendado. Esta metodología permite organizar el acceso a memoria de forma estructurada y evita conflictos durante las operaciones de escritura.

El diagrama también permite observar que el módulo funciona de manera completamente combinacional, ya que las salidas dependen únicamente del valor actual de la dirección de entrada. Gracias a esta estructura, la FSM puede controlar de manera precisa qué posición de memoria será utilizada durante cada etapa del procesamiento del sistema.

➤ Entradas y Salidas

SEÑAL	ANCHO	DIRECCIÓN	DESCRIPCIÓN
addr	4 bits	Entrada	Dirección generada por la FSM principal
sel_rssi_ch1	1 bit	Salida	Activa posición de RSSI – canal 1
sel_occ_ch1	1 bit	Salida	Activa posición de ocupación – canal 1
sel_state_ch1	1 bit	Salida	Activa posición de estado – canal 1
sel_rssi_ch6	1 bit	Salida	Activa posición de RSSI – canal 6
sel_occ_ch6	1 bit	Salida	Activa posición de ocupación – canal 6
sel_state_ch6	1 bit	Salida	Activa posición de estado – canal 6
sel_rssi_ch11	1 bit	Salida	Activa posición de RSSI – canal 11
sel_occ_ch11	1 bit	Salida	Activa posición de ocupación – canal 11
sel_state_ch11	1 bit	Salida	Activa posición de estado – canal 11
sel_recommended	1 bit	Salida	Activa posición del canal recomendado

Tabla 7.- Entradas y Salidas de Módulo decodificador de direcciones

➤ Pseudocódigo

MÓDULO: address_decoder

ENTRADA: addr [3:0]

SALIDAS: sel_rssi_ch1, sel_occ_ch1, sel_state_ch1,
sel_rssi_ch6, sel_occ_ch6, sel_state_ch6,
sel_rssi_ch11, sel_occ_ch11, sel_state_ch11,
sel_recommended

PROCESO COMBINACIONAL:

Poner todas las señales de selección en 0

Según el valor de addr:

0x00 → sel_rssi_ch1 = 1
 0x01 → sel_occ_ch1 = 1
 0x02 → sel_state_ch1 = 1
 0x04 → sel_rssi_ch6 = 1
 0x05 → sel_occ_ch6 = 1
 0x06 → sel_state_ch6 = 1
 0x08 → sel_rssi_ch11 = 1
 0x09 → sel_occ_ch11 = 1
 0x0A → sel_state_ch11 = 1
 0x0F → sel_recommended = 1
 Otro → todas las señales permanecen en 0

Interpretación:

El pseudocódigo muestra un esquema de selección exclusiva donde todas las señales son inicializadas en cero antes de evaluar la dirección recibida. Posteriormente, dependiendo del valor de *addr*, únicamente una salida es activada. Esto evita activaciones simultáneas y asegura un acceso ordenado a las posiciones de memoria.

➤ Código HDL(Verilog)

```

module address_decoder(
  input [3:0] addr,
  output reg sel_rssi_ch1,
  output reg sel_occ_ch1,
  output reg sel_state_ch1,
  output reg sel_rssi_ch6,
  output reg sel_occ_ch6,
  output reg sel_state_ch6,
  output reg sel_rssi_ch11,
  output reg sel_occ_ch11,
  output reg sel_state_ch11,
  output reg sel_recommended
);
  always @(*) begin
    // Inicializar todas las señales en 0
    sel_rssi_ch1 = 0;
    sel_occ_ch1 = 0;
    sel_state_ch1 = 0;
    sel_rssi_ch6 = 0;
    sel_occ_ch6 = 0;
    sel_state_ch6 = 0;
    sel_rssi_ch11 = 0;
    sel_occ_ch11 = 0;
    sel_state_ch11 = 0;
    sel_recommended = 0;

    // Decodificar dirección
    case (addr)
      4'h0: sel_rssi_ch1 = 1;
      4'h1: sel_occ_ch1 = 1;
      4'h2: sel_state_ch1 = 1;
      4'h4: sel_rssi_ch6 = 1;
      4'h5: sel_occ_ch6 = 1;
      4'h6: sel_state_ch6 = 1;
      4'h8: sel_rssi_ch11 = 1;
      4'h9: sel_occ_ch11 = 1;
    endcase
  end

```

```

4'hA: sel_state_ch11 = 1;
4'hF: sel_recommended = 1;
default: begin
    // No activar ninguna señal
    sel_rssi_ch1 = 0;
    sel_occ_ch1 = 0;
    sel_state_ch1 = 0;
    sel_rssi_ch6 = 0;
    sel_occ_ch6 = 0;
    sel_state_ch6 = 0;
    sel_rssi_ch11 = 0;
    sel_occ_ch11 = 0;
    sel_state_ch11 = 0;
    sel_recommended = 0;
end
endcase
end
endmodule

```

Interpretación:

El módulo utiliza una estructura case para implementar el proceso de decodificación de direcciones. Antes de realizar la comparación, todas las señales de selección son reinicializadas a cero, evitando que permanezcan activas entre evaluaciones consecutivas. Cada caso hexadecimal activa una línea específica asociada a una métrica del sistema, mientras que el bloque default garantiza estabilidad ante direcciones inválidas.

3.4. Módulo memoria de métricas

El módulo de memoria de métricas implementa un banco de registros de 16 posiciones, cada una de 8 bits, que actúa como almacenamiento centralizado del sistema. Su función es guardar los valores de RSSI, ocupación y estado clasificado de cada uno de los tres canales analizados, así como el canal recomendado al final del ciclo de evaluación. La escritura se realiza de forma síncrona, es decir, los datos se almacenan en el flanco positivo del reloj cuando la señal de habilitación de escritura está activa.

La lectura, en cambio, es asíncrona o combinacional: el dato almacenado en la dirección indicada se presenta en la salida de forma inmediata, sin necesidad de esperar un ciclo de reloj. La señal de reset inicializa todas las posiciones del banco en cero. Esta arquitectura permite que la FSM principal pueda escribir y leer métricas de manera controlada según el estado en que se encuentre, siguiendo el mapa de memoria definido para el sistema.

➤ **Diagrama de bloques**

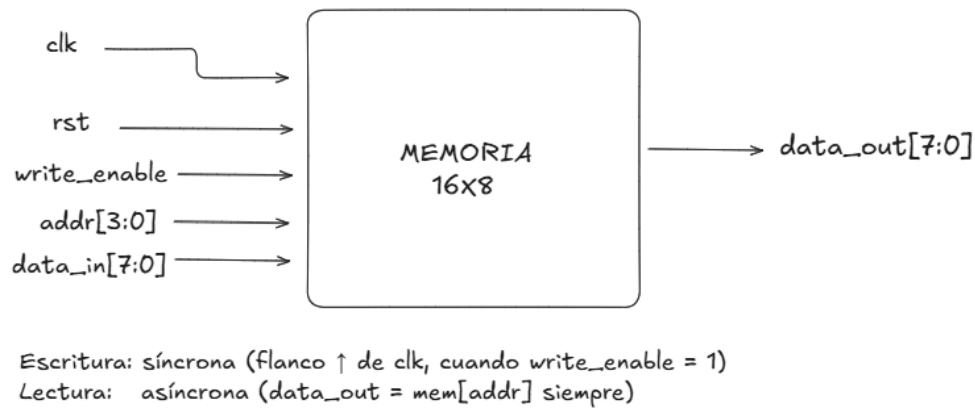


Figura 8.- Diagrama de bloque de Módulo de memorias de métricas

El diagrama de bloques del módulo de memoria representa la unidad encargada de almacenar temporalmente las métricas procesadas por el sistema digital. El bloque recibe señales de control como *clk*, *rst* y *write_enable*, además de la dirección *addr* y el dato de entrada *data_in*, los cuales permiten controlar el acceso a las distintas posiciones de memoria internas.

La estructura mostrada en el diagrama evidencia la presencia de una memoria direccionada compuesta por múltiples registros organizados internamente. Durante las operaciones de escritura, la dirección selecciona la posición específica donde será almacenado el dato recibido. Asimismo, la señal *write_enable* determina cuándo la información puede ser registrada dentro de la memoria.

El diagrama también muestra la existencia de una salida *data_out*, utilizada para recuperar el contenido almacenado en la dirección seleccionada. Este mecanismo permite que la FSM pueda acceder posteriormente a las métricas guardadas para realizar operaciones de comparación y selección del canal recomendado. La organización presentada facilita el almacenamiento secuencial y estructurado de la información generada por el sistema.

➤ **Entradas y Salidas**

SEÑAL	ANCHO	DIRECCIÓN	DESCRIPCIÓN
clk	1 bit	Entrada	Reloj del sistema (flanco positivo)
rst	1 bit	Entrada	Reset síncrono — inicializa toda la memoria en cero
write_enable	1 bit	Entrada	Habilita la escritura en la posición indicada por addr
addr	4 bits	Entrada	Dirección de la posición de memoria a acceder

data_in	8 bits	Entrada	Dato a almacenar cuando write_enable = 1
data_out	8 bits	Salida	Dato leído de la posición indicada por addr

Tabla 8.- Entradas y Salida del Módulo memoria de métricas

El módulo de memoria implementa físicamente el mapa de direcciones definido previamente en la sección 2.4.

➤ Pseudocódigo

MÓDULO: metric_memory

ENTRADAS: clk, rst, write_enable, addr [3:0], data_in [7:0]

SALIDA: data_out [7:0]

DEFINICIÓN: mem[0..15] — arreglo de 16 registros de 8 bits

PROCESO SÍNCRONO (en flanco positivo de clk):

Si rst = 1:

Para i = 0 hasta 15: mem[i] ← 0

Si no y write_enable = 1:

mem[addr] ← data_in

PROCESO COMBINACIONAL (lectura asíncrona):

data_out = mem[addr]

Interpretación:

El pseudocódigo diferencia claramente dos operaciones fundamentales: escritura síncrona y lectura combinacional. La escritura únicamente ocurre durante el flanco positivo del reloj y cuando la señal *write_enable* está activa, mientras que la lectura permite acceder inmediatamente al contenido almacenado en la dirección seleccionada.

➤ Código HDL (Verilog)

```
module metric_memory(
    input clk,
    input rst,
    input write_enable,
    input [3:0] addr,
    input [7:0] data_in,
    output reg [7:0] data_out
);
    // Declaración de memoria: 16 posiciones de 8 bits
    reg [7:0] memory [0:15];
    integer i;

    // Proceso de escritura síncrona
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            // Inicializar toda la memoria a cero
```

```

        for (i = 0; i < 16; i = i + 1)
            memory[i] <= 8'd0;
        end else begin
            if (write_enable)
                memory[addr] <= data_in;
            end
        end
    end

    // Proceso de Lectura combinacional
    always @(*) begin
        data_out = memory[addr];
    end
endmodule

```

Interpretación:

El código HDL implementa una memoria direccionada utilizando un arreglo interno de registros denominado *memory*. El bloque secuencial controlado por *posedge clk* permite almacenar datos de forma sincronizada, mientras que el proceso de lectura combinacional entrega continuamente el contenido asociado a la dirección activa. El ciclo *for* empleado durante el reset inicializa completamente la memoria, garantizando un estado conocido al iniciar el sistema.

3.5. Módulo comparador de canales

El módulo comparador de canales es el encargado de determinar cuál de los tres canales analizados presenta el menor porcentaje de ocupación, y por tanto, el menor nivel de congestión. A partir de los valores de ocupación de los canales 1, 6 y 11, el módulo realiza una comparación combinacional y entrega como resultado el número del canal recomendado, codificado como un valor de 8 bits.

En caso de empate entre dos o más canales con la misma ocupación mínima, el módulo aplica un criterio de prioridad fija: se prioriza el canal 1 sobre el canal 6, y este último sobre el canal 11. Este comportamiento determinístico garantiza que la salida sea siempre unívoca. El módulo opera de forma combinacional y su resultado es capturado por la FSM principal en el estado COMPARE antes de ser almacenado en memoria.

➤ Diagrama de bloques

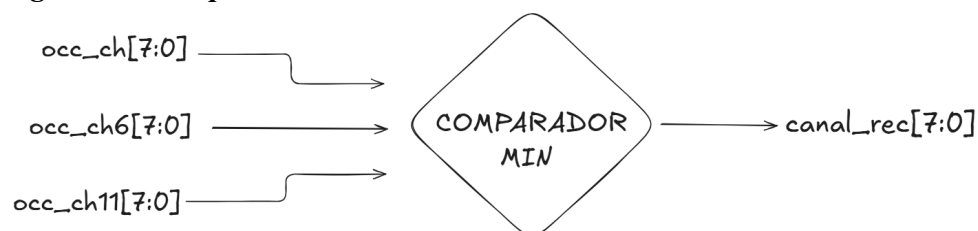


Figura 9.- Diagrama de bloque de Módulo de comparador de canales

El diagrama de bloques del módulo comparador de canales representa el proceso de evaluación de ocupación entre los tres canales inalámbricos analizados. El módulo recibe como entradas los valores *occ_ch1*, *occ_ch6* y *occ_ch11*, correspondientes a los porcentajes de ocupación de cada canal monitoreado por el sistema.

Dentro del bloque comparador se realiza una evaluación combinatorial para determinar cuál de los canales presenta menor nivel de congestión. A partir de esta comparación, el módulo genera como salida el valor *canal_recomendado*, el cual corresponde al canal más adecuado para la comunicación inalámbrica según las métricas procesadas.

El diagrama también evidencia que el sistema implementa una lógica determinística de prioridad en caso de empate entre ocupaciones similares. Esta característica garantiza que el módulo siempre produzca una única salida válida, evitando ambigüedades durante el proceso de recomendación. Debido a que el bloque opera de forma combinatorial, cualquier cambio en las métricas de entrada actualiza inmediatamente el canal recomendado.

➤ *Entrada y Salida*

C	ANCHO	DIRECCIÓN	DESCRIPCIÓN
occ_ch1	8 bits	Entrada	Ocupación del canal 1 (0 a 100)
occ_ch6	8 bits	Entrada	Ocupación del canal 6 (0 a 100)
occ_ch11	8 bits	Entrada	Ocupación del canal 11 (0 a 100)
canal_recomendado	8 bits	Salida	Número del canal con menor ocupación (1, 6 u 11)

Tabla 9.- Entradas y Salida de Módulo comparador de canales

➤ *Pseudocódigo*

MÓDULO: channel_comparator

ENTRADAS: *occ_ch1* [7:0], *occ_ch6* [7:0], *occ_ch11* [7:0]

SALIDA: *canal_recomendado* [7:0]

PROCESO COMBINACIONAL:

Si $occ_ch1 \leq occ_ch6$ Y $occ_ch1 \leq occ_ch11$:

canal_recomendado ← 1

Si no, si $occ_ch6 < occ_ch1$ Y $occ_ch6 \leq occ_ch11$:

canal_recomendado ← 6

Si no:

canal_recomendado ← 11

(En caso de empate se prioriza: canal 1 > canal 6 > canal 11)

Interpretación:

El pseudocódigo realiza una comparación secuencial entre los niveles de ocupación de los tres canales analizados. El algoritmo prioriza el canal con menor congestión y aplica un criterio determinístico en caso de igualdad, evitando resultados ambiguos durante la selección del canal recomendado.

➤ **Código HDL (Verilog)**

```
module channel_comparator(
    input [7:0] occ_ch1,
    input [7:0] occ_ch6,
    input [7:0] occ_ch11,
    output reg [7:0] canal_recomendado
);
    // Selecciona el canal con menor ocupación
    // En caso de empate, prioriza CH1, luego CH6, luego CH11

    always @(*) begin
        if ((occ_ch1 <= occ_ch6) && (occ_ch1 <= occ_ch11))
            canal_recomendado = 8'd1;
        else if ((occ_ch6 < occ_ch1) && (occ_ch6 <= occ_ch11))
            canal_recomendado = 8'd6;
        else
            canal_recomendado = 8'd11;
    end
endmodule
```

Interpretación:

El código implementa comparaciones combinatoriales entre las entradas *occ_ch1*, *occ_ch6* y *occ_ch11*. La prioridad establecida en las condiciones permite resolver empates favoreciendo primero al canal 1 y posteriormente al canal 6. Debido al uso de *always @(*)*, la recomendación del canal se actualiza inmediatamente ante cualquier modificación en los niveles de ocupación.

3.6. Módulo principal integrado

El módulo principal integrado, denominado *ism_analyzer_top*, actúa como contenedor jerárquico del sistema y es responsable de coordinar el flujo de operación entre todos los submódulos. Su función es instanciar y conectar el decodificador de direcciones, la memoria de métricas, los tres módulos LUT de decisión y el comparador de canales, controlando la secuencia de operaciones mediante una máquina de estados finitos (FSM) interna.

La FSM del módulo principal recorre catorce estados de forma secuencial: comienza en el estado de espera (IDLE), captura las métricas de entrada, escribe los valores de RSSI, ocupación y estado de cada canal en la memoria, ejecuta la comparación, almacena el canal recomendado y finalmente activa la señal de validación de salida (*output_valid*). Todo el

ciclo demanda aproximadamente 13 ciclos de reloj desde la activación de `data_valid`, lo que equivale a una latencia de decisión de 260 ns con un reloj de 50 MHz.

Las señales de control internas —`write_enable`, `addr` y `data_in`— son generadas por la FSM y dirigen la escritura en memoria estado por estado. La arquitectura modular garantiza que cada componente pueda ser verificado de forma independiente y que el sistema completo sea fácilmente escalable.

➤ Diagrama de bloques

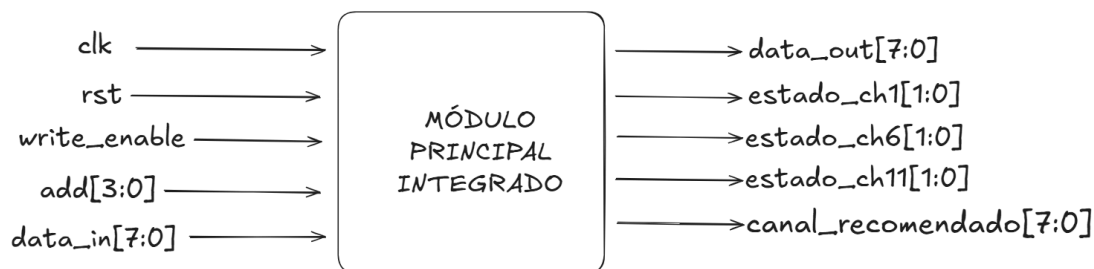


Figura 10.- Diagrama de bloque del Módulo principal integrado

El diagrama de bloques del módulo principal integrado representa la arquitectura general del sistema digital implementado en FPGA. En esta estructura se observa la interconexión entre todos los submódulos desarrollados, incluyendo los módulos LUT de decisión, la memoria de métricas, el comparador de canales y la FSM principal encargada del control secuencial del sistema.

Las señales de entrada corresponden a las métricas RSSI y ocupación provenientes de los canales 1, 6 y 11, además de las señales de control del sistema como reloj, reset y validación de datos. Estas métricas son inicialmente capturadas y posteriormente enviadas hacia los módulos LUT, donde se realiza la clasificación del nivel de ocupación de cada canal analizado.

El diagrama también evidencia que la FSM coordina el flujo completo de operación del sistema, controlando las señales de escritura, direccionamiento y almacenamiento de métricas dentro de la memoria interna. Posteriormente, la información almacenada es enviada al comparador de canales, el cual determina cuál presenta menor congestión y genera el canal recomendado como resultado final del procesamiento.

Finalmente, el diagrama muestra que el módulo principal actúa como unidad central de integración y control, permitiendo coordinar de forma secuencial el procesamiento de datos, la clasificación de estados, el acceso a memoria y la generación de resultados dentro de la arquitectura digital completa.

➤ **Entrada y Salida**

SEÑAL	ANCHO	DIRECCIÓN	DESCRIPCIÓN
clk	1 bit	Entrada	Reloj del sistema
rst	1 bit	Entrada	Reset activo alto
data_valid	1 bit	Entrada	Indica que las métricas de entrada son válidas
rss_i_ch1	8 bits	Entrada	RSSI del canal 1
occ_ch1	8 bits	Entrada	Ocupación del canal 1
rss_i_ch6	8 bits	Entrada	RSSI del canal 6
occ_ch6	8 bits	Entrada	Ocupación del canal 6
rss_i_ch11	8 bits	Entrada	RSSI del canal 11
occ_ch11	8 bits	Entrada	Ocupación del canal 11
estado_ch1	2 bits	Salida	Estado clasificado del canal 1
estado_ch6	2 bits	Salida	Estado clasificado del canal 6
estado_ch11	2 bits	Salida	Estado clasificado del canal 11
canal_recomendado	8 bits	Salida	Canal sugerido por el sistema (1, 6 u 11)
output_valid	1 bit	Salida	Indica que la decisión está lista

Tabla 10.- Entrada y Salida de Módulo principal integrado

➤ **Secuencia de estados de la FSM interna**

ESTADO	ACCIÓN REALIZADA	WRITE_ENABLE	DIRECCIÓN
IDLE	Espera data_valid = 1	0	—
CAPTURE	Captura las métricas en registros internos	0	—
WRITE_CH1_RSSI	Escribe RSSI del canal 1 en memoria	1	0x00
WRITE_CH1_OCC	Escribe ocupación del canal 1	1	0x01
WRITE_CH1_STATE	Escribe estado del canal 1	1	0x02
WRITE_CH6_RSSI	Escribe RSSI del canal 6	1	0x04
WRITE_CH6_OCC	Escribe ocupación del canal 6	1	0x05
WRITE_CH6_STATE	Escribe estado del canal 6	1	0x06
WRITE_CH11_RSSI	Escribe RSSI del canal 11	1	0x08
WRITE_CH11_OCC	Escribe ocupación del canal 11	1	0x09

WRITE_CH11_STATE	Escribe estado del canal 11	1	0x0A
COMPARE	Obtiene el canal con menor ocupación	0	—
WRITE_RESULT	Escribe el canal recomendado en memoria	1	0x0F
OUTPUT_STATE	Activa output_valid y regresa a IDLE	0	—

Tabla 11.- Secuencia de estados de la FSM interna

➤ **Pseudocódigo**

MÓDULO: ism_analyzer_top

INICIALIZACIÓN (rst = 1):

state ← IDLE; todos los registros ← 0

CICLO PRINCIPAL (en cada flanco positivo de clk):

IDLE: Si data_valid = 1 → ir a CAPTURE

CAPTURE: Cargar rssi_chX y occ_chX en registros internos

WRITE_CH1_RSSI: addr=0x00, data_in=rssi1_reg, write_enable=1

WRITE_CH1_OCC: addr=0x01, data_in=occ1_reg, write_enable=1

WRITE_CH1_STATE: addr=0x02, data_in=estado1, write_enable=1

WRITE_CH6_RSSI: addr=0x04, data_in=rssi6_reg, write_enable=1

WRITE_CH6_OCC: addr=0x05, data_in=occ6_reg, write_enable=1

WRITE_CH6_STATE: addr=0x06, data_in=estado6, write_enable=1

WRITE_CH11_RSSI: addr=0x08, data_in=rssi11_reg, write_enable=1

WRITE_CH11_OCC: addr=0x09, data_in=occ11_reg, write_enable=1

WRITE_CH11_STATE: addr=0x0A, data_in=estado11, write_enable=1

COMPARE: canal_recomendado ← resultado del comparador

WRITE_RESULT: addr=0x0F, data_in=canal_recomendado, write_enable=1

OUTPUT_STATE: output_valid ← 1; regresar a IDLE

Interpretación:

El pseudocódigo refleja el comportamiento secuencial de la FSM principal, donde cada estado ejecuta una tarea específica dentro del procesamiento de métricas. La secuencia inicia con la captura de datos, continúa con la escritura organizada en memoria y finaliza con la generación del canal recomendado y la activación de la señal de salida válida.

➤ **Código HDL (Verilog)**

```
module ism_analyzer_top(
    input clk,
    input rst,
    input data_valid,
    input [7:0] rssi_ch1,
    input [7:0] occ_ch1,
    input [7:0] rssi_ch6,
    input [7:0] occ_ch6,
```

```

input [7:0] rssi_ch11,
input [7:0] occ_ch11,
output [1:0] estado_ch1,
output [1:0] estado_ch6,
output [1:0] estado_ch11,
output reg [7:0] canal_recomendado,
output reg output_valid,
output reg [3:0] state,
output reg [3:0] addr,
output reg write_enable,
output reg [7:0] data_in,
output [7:0] data_out
);

// Definición de estados
localparam IDLE = 4'd0;
localparam CAPTURE = 4'd1;
localparam WRITE_CH1_RSSI = 4'd2;
localparam WRITE_CH1_OCC = 4'd3;
localparam WRITE_CH1_STATE = 4'd4;
localparam WRITE_CH6_RSSI = 4'd5;
localparam WRITE_CH6_OCC = 4'd6;
localparam WRITE_CH6_STATE = 4'd7;
localparam WRITE_CH11_RSSI = 4'd8;
localparam WRITE_CH11_OCC = 4'd9;
localparam WRITE_CH11_STATE = 4'd10;
localparam COMPARE = 4'd11;
localparam WRITE_RESULT = 4'd12;
localparam OUTPUT_STATE = 4'd13;

// Registros internos
reg [7:0] rssi1_reg, occ1_reg;
reg [7:0] rssi6_reg, occ6_reg;
reg [7:0] rssi11_reg, occ11_reg;
wire [1:0] estado1_wire, estado6_wire, estado11_wire;
wire [7:0] recommended_wire;

// Instanciación de módulos
decision_lut LUT1 (.ocupacion(occ1_reg), .estado(estado1_wire));
decision_lut LUT6 (.ocupacion(occ6_reg), .estado(estado6_wire));
decision_lut LUT11(.ocupacion(occ11_reg), .estado(estado11_wire));

channel_comparator COMP(
    .occ_ch1(occ1_reg),
    .occ_ch6(occ6_reg),
    .occ_ch11(occ11_reg),
    .canal_recomendado(recommended_wire)
);

metric_memory MEM(
    .clk(clk),
    .rst(rst),
    .write_enable(write_enable),
    .addr(addr),
    .data_in(data_in),
    .data_out(data_out)
);

// Asignación de salidas de estado

```

```

assign estado_ch1 = estado1_wire;
assign estado_ch6 = estado6_wire;
assign estado_ch11 = estado11_wire;

// Máquina de estados finitos
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        addr <= 4'd0;
        data_in <= 8'd0;
        write_enable <= 1'b0;
        output_valid <= 1'b0;
        canal_recomendado <= 8'd0;
        rssi1_reg <= 8'd0;
        occ1_reg <= 8'd0;
        rssi6_reg <= 8'd0;
        occ6_reg <= 8'd0;
        rssi11_reg <= 8'd0;
        occ11_reg <= 8'd0;
    end else begin
        write_enable <= 1'b0;
        output_valid <= 1'b0;

        case (state)
            IDLE: begin
                if (data_valid)
                    state <= CAPTURE;
            end

            CAPTURE: begin
                rssi1_reg <= rssi_ch1;
                occ1_reg <= occ_ch1;
                rssi6_reg <= rssi_ch6;
                occ6_reg <= occ_ch6;
                rssi11_reg <= rssi_ch11;
                occ11_reg <= occ_ch11;
                state <= WRITE_CH1_RSSI;
            end

            WRITE_CH1_RSSI: begin
                addr <= 4'h0;
                data_in <= rssi1_reg;
                write_enable <= 1'b1;
                state <= WRITE_CH1_OCC;
            end

            WRITE_CH1_OCC: begin
                addr <= 4'h1;
                data_in <= occ1_reg;
                write_enable <= 1'b1;
                state <= WRITE_CH1_STATE;
            end

            WRITE_CH1_STATE: begin
                addr <= 4'h2;
                data_in <= {6'b000000, estado1_wire};
                write_enable <= 1'b1;
                state <= WRITE_CH6_RSSI;
            end
        endcase
    end
end

```

```

WRITE_CH6_RSSI: begin
    addr <= 4'h4;
    data_in <= rssi6_reg;
    write_enable <= 1'b1;
    state <= WRITE_CH6_OCC;
end

WRITE_CH6_OCC: begin
    addr <= 4'h5;
    data_in <= occ6_reg;
    write_enable <= 1'b1;
    state <= WRITE_CH6_STATE;
end

WRITE_CH6_STATE: begin
    addr <= 4'h6;
    data_in <= {6'b000000, estado6_wire};
    write_enable <= 1'b1;
    state <= WRITE_CH11_RSSI;
end

WRITE_CH11_RSSI: begin
    addr <= 4'h8;
    data_in <= rssi11_reg;
    write_enable <= 1'b1;
    state <= WRITE_CH11_OCC;
end

WRITE_CH11_OCC: begin
    addr <= 4'h9;
    data_in <= occ11_reg;
    write_enable <= 1'b1;
    state <= WRITE_CH11_STATE;
end

WRITE_CH11_STATE: begin
    addr <= 4'hA;
    data_in <= {6'b000000, estado11_wire};
    write_enable <= 1'b1;
    state <= COMPARE;
end

COMPARE: begin
    canal_recomendado <= recommended_wire;
    state <= WRITE_RESULT;
end

WRITE_RESULT: begin
    addr <= 4'hF;
    data_in <= canal_recomendado;
    write_enable <= 1'b1;
    state <= OUTPUT_STATE;
end

OUTPUT_STATE: begin
    output_valid <= 1'b1;
    state <= IDLE;
end

```

```

        default: begin
            state <= IDLE;
        end
    endcase
end
end
endmodule

```

Interpretación:

El módulo principal integrado implementa la arquitectura central del sistema digital y actúa como unidad de control encargada de coordinar el funcionamiento de todos los submódulos desarrollados. A través de este bloque se integran el decodificador de direcciones, la memoria de métricas, los módulos LUT de decisión y el comparador de canales, permitiendo ejecutar de manera secuencial el procesamiento completo de las métricas recibidas desde los canales inalámbricos analizados. El módulo recibe como entradas las señales RSSI y ocupación de los canales 1, 6 y 11, además de las señales de reloj, reset y validación de datos, mientras que sus salidas corresponden a los estados clasificados de cada canal y al canal recomendado generado por el sistema.

Dentro del código HDL se define inicialmente el conjunto de entradas, salidas y señales internas necesarias para la operación del sistema. Las señales de entrada permiten ingresar las métricas provenientes de los módulos externos, mientras que las salidas entregan el resultado final del procesamiento realizado por la FPGA. Asimismo, el diseño utiliza registros internos para almacenar temporalmente las métricas capturadas antes de ser escritas en memoria. Esta organización permite mantener un flujo ordenado de información y facilita la sincronización entre los distintos procesos internos implementados en el sistema digital.

Posteriormente, el código establece la definición de estados de la máquina de estados finitos mediante constantes *localparam*. Cada estado representa una operación específica dentro del ciclo de funcionamiento del sistema. Entre estas operaciones se encuentran la espera de datos válidos, la captura de métricas, la escritura secuencial de información en memoria, la comparación de ocupaciones y la generación del resultado final. Esta estructura secuencial permite dividir el procesamiento completo en etapas controladas y sincronizadas con el reloj del sistema.

El módulo también incorpora la instanciación de los submódulos encargados de realizar funciones específicas dentro de la arquitectura general. Se utilizan tres módulos *decision_lut*, uno para cada canal analizado, permitiendo clasificar individualmente el nivel de ocupación de cada canal inalámbrico. Del mismo modo, se integra el módulo *channel_comparator*, cuya función consiste en determinar cuál de los canales presenta menor congestión. Finalmente, el

módulo *metric_memory* permite almacenar las métricas procesadas utilizando direcciones específicas controladas por la FSM principal. Esta integración modular facilita la reutilización y verificación independiente de cada componente del sistema.

La lógica secuencial principal se implementa mediante el bloque *always @(posedge clk or posedge rst)*, el cual controla el comportamiento de la FSM sincronizada con el reloj del sistema. Cuando la señal de reset se activa, todos los registros internos, señales de control y variables de estado son inicializados en cero, garantizando que el sistema comience desde una condición estable y conocida. Una vez finalizado el reinicio, la máquina de estados inicia su secuencia normal de operación a partir del estado *IDLE*.

Durante el estado *CAPTURE*, el sistema almacena temporalmente las métricas RSSI y ocupación de cada canal dentro de registros internos. Posteriormente, la FSM avanza por múltiples estados de escritura donde se activan las señales *write_enable*, *addr* y *data_in* para registrar ordenadamente la información en memoria. Cada estado *WRITE* se encarga de almacenar una métrica específica, permitiendo organizar las direcciones correspondientes al RSSI, ocupación y estado clasificado de cada canal analizado. Esta metodología garantiza un acceso estructurado y secuencial a los datos almacenados.

Después del almacenamiento de métricas, el sistema ingresa al estado *COMPARE*, donde se obtiene el resultado generado por el comparador de canales. En esta etapa se determina cuál de los canales posee menor nivel de ocupación y, por lo tanto, representa la mejor opción de comunicación inalámbrica. El resultado obtenido es posteriormente almacenado en memoria durante el estado *WRITE_RESULT*, utilizando la dirección reservada para el canal recomendado dentro del mapa de memoria definido previamente.

Finalmente, en el estado *OUTPUT_STATE*, la señal *output_valid* es activada para indicar que el procesamiento del sistema ha concluido correctamente y que el canal recomendado se encuentra disponible en la salida. Una vez entregado el resultado final, la FSM retorna nuevamente al estado *IDLE*, quedando preparada para iniciar un nuevo ciclo de procesamiento cuando se reciban nuevas métricas válidas desde el sistema de adquisición. De esta manera, el módulo principal integrado coordina completamente el flujo de operación de la arquitectura digital implementada en FPGA.

4. Simulación

Se realizó una simulación funcional completa del sistema en ModelSim para validar el comportamiento de la FSM, el acceso a memoria, el direccionamiento, la clasificación mediante LUT y la selección automática del canal menos congestionado.

4.1. Objetivo de la simulación

- Transición correcta de estados de la FSM.
- Almacenamiento de métricas en memoria.
- Funcionamiento del decodificador de direcciones.
- Clasificación de canales mediante LUT, selección automática del canal recomendado

4.2. Plan de simulación

La simulación se ejecutó mediante un testbench en Verilog HDL utilizando señales de prueba para RSSI y ocupación de los canales 1, 6 y 11. Se evaluaron las siguientes etapas:

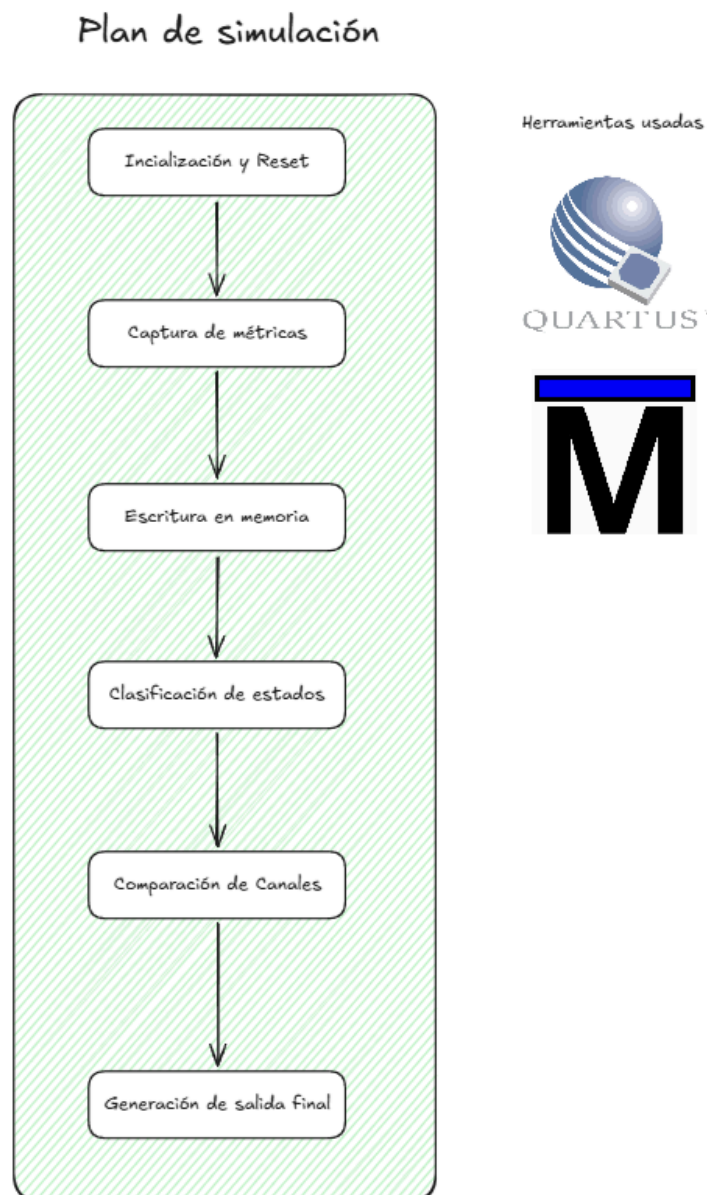
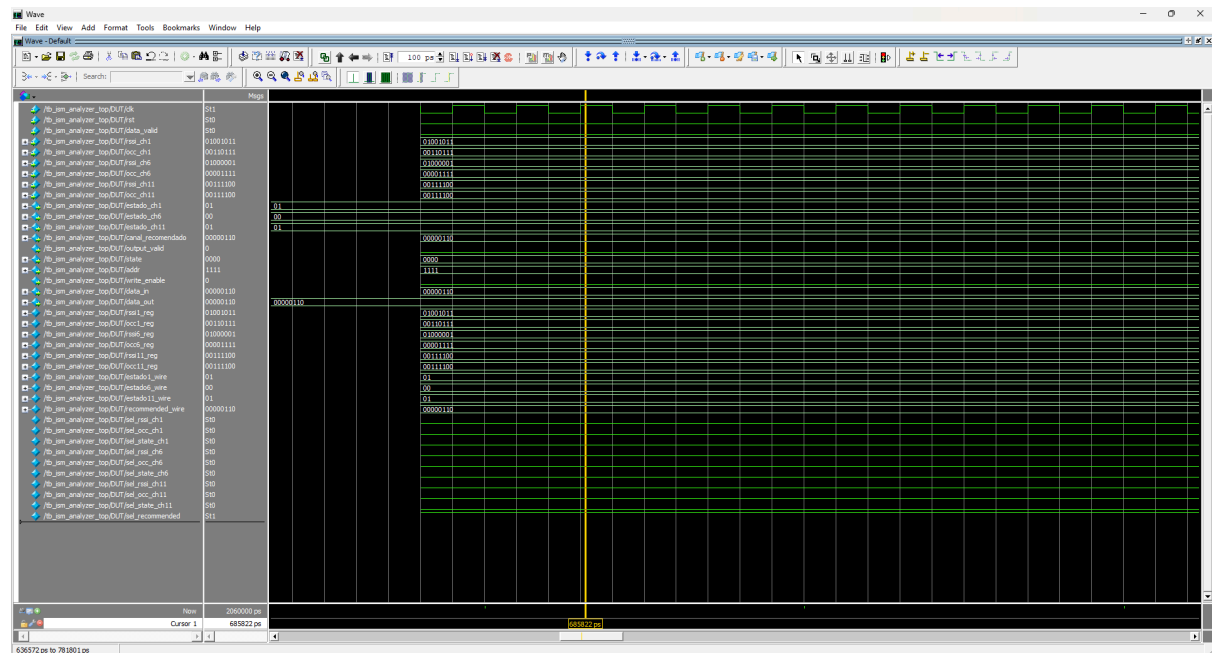
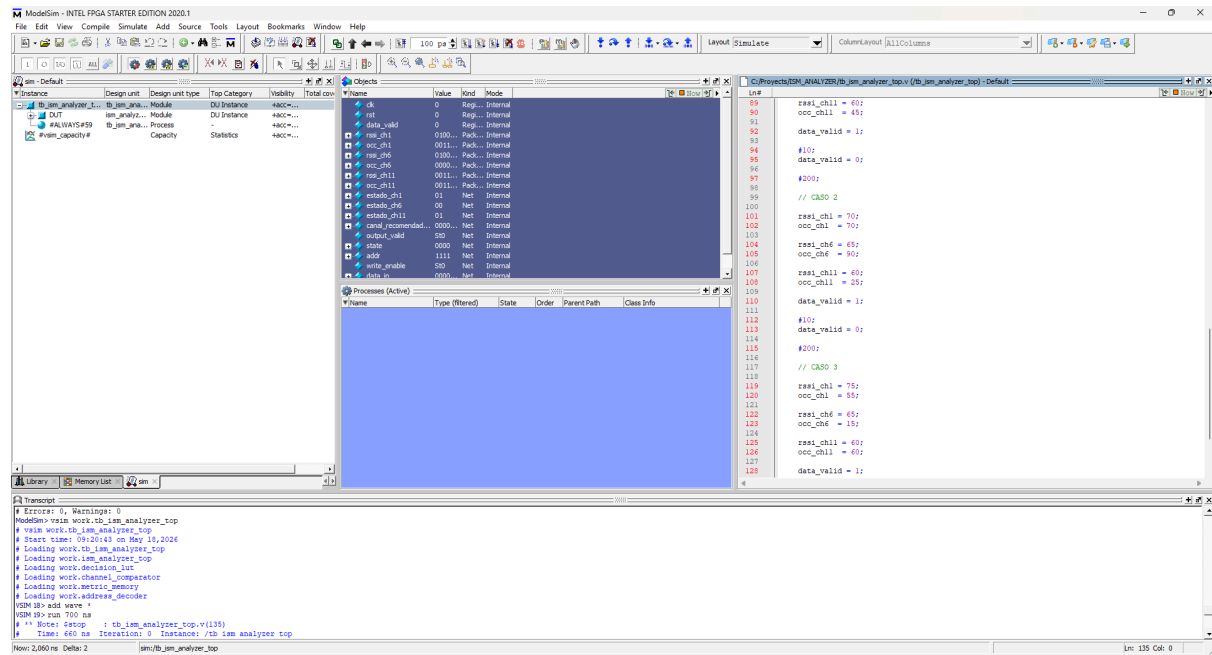


Figura 11.- Plan de simulación



```

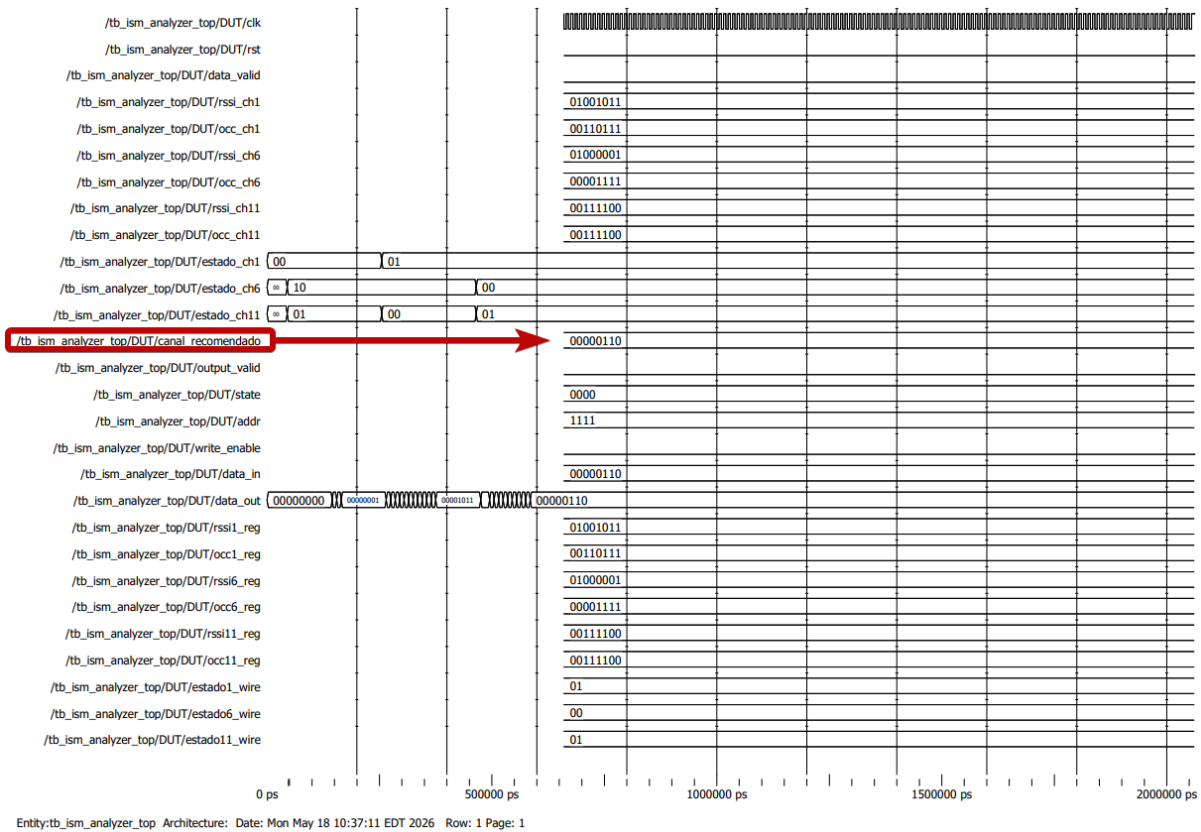
// Verilog code for a digital circuit, likely a FSM or a data path component.
// The code includes module declarations, signal assignments, and a large case statement for a FSM.
// The code is written in Verilog and is intended for synthesis into a digital circuit.

module fsm_analyzer_top (
    rst,
    clk,
    state_ch1,
    state_ch6,
    state_ch11,
    canal_recomendado,
    output_valid
);
    // ... (rest of the code) ...
endmodule

```

4.3. Señales observadas

Señal	Función
clk	reloj principal
rst	reinicio del sistema
state	estado actual de la FSM
addr	dirección de memoria
write_enable	habilitación de escritura
data_in	dato de entrada a memoria
data_out	dato leído de memoria
estado_ch1	estado del canal 1
estado_ch6	estado del canal 6
estado_ch11	estado del canal 11
canal_recomendado	canal seleccionado
output_valid	validación de salida



4.4. Casos de prueba

Canal	RSSI	Ocupación
CH1	75	55
CH6	65	15
CH11	60	60

5. Conclusiones técnicas

El diseño propuesto cumple con el requisito central del proyecto, al integrar una FSM, un decodificador de direcciones y una memoria direccionada dentro del flujo activo de datos. Además, la simulación permite validar la escritura de métricas por canal, la clasificación mediante LUT y la selección del canal con menor ocupación. Asimismo, el sistema se encuentra preparado para una futura etapa de implementación física, donde las métricas simuladas podrán ser reemplazadas por datos provenientes de módulos ESP32-S3. La FPGA se justifica como coprocesador digital multicanal, ya

que permite controlar la memoria, procesar métricas y generar una decisión con latencia determinística.

6. Referencias

- [1] Teguig, D., Azzaz, E., Militaire, P., Bp17, B., & el Bahri-Algiers, A. (2018). FPGA implementation of Spectrum Sensing methods for Cognitive Radio. *2018 International Symposium on Networks, Computers and Communications (ISNCC)*. <https://doi.org/10.1109/ISNCC.2018.8530971>
- [2] Liu, H., Wei, Z., Zhang, H., Li, B., & Zhao, C. (2022). Tiny Machine Learning (Tiny-ML) for Efficient Channel Estimation and Signal Detection. *IEEE Transactions on Vehicular Technology*, 71(6), 6795–6800. <https://doi.org/10.1109/TVT.2022.3163786>
- [3] Abdelghany, A., Uguen, B., Moy, C., & Lemur, D. (2022). Decentralized Adaptive Spectrum Learning in Wireless IoT Networks Based on Channel Quality Information. *IEEE Internet of Things Journal*, 9(20), 19660–19669. <https://doi.org/10.1109/JIOT.2022.3167016>
- [4] Forleo, M., Pau, D. pietro, & Piazzese, N. I. (2022). Tiny Neural Networks for ISM band demodulation. *MELECON 2022 - IEEE Mediterranean Electrotechnical Conference, Proceedings*, 752–757. <https://doi.org/10.1109/MELECON53508.2022.984>

